

# Project Architecture: How it works

This document explains how the MCQ Solver AI extension functions "under the hood."

## 🧩 Components: How things work together

The extension is divided into three main parts that work together like a team:

| Component    | Responsibility                                                                                                                      | File Location                                       |
|--------------|-------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------|
| The Eyes     | <b>Content Script:</b> Scans the webpage, finds the MCQ text/options, and displays the floating popup UI.                           | <code>content/content.js</code>                     |
| The Brain    | <b>Background Service Worker:</b> Handles the "Orchestration" — it decides which AI to call, handles errors, and remembers history. | <code>background/background.js</code>               |
| The Registry | <b>Options &amp; Storage:</b> Securely stores your private API keys and extension settings.                                         | <code>options/</code> & <code>chrome.storage</code> |

## ⚙️ The "Backend" Orchestration Logic

Code Location: `background/background.js`

When you press **Ctrl+Shift+Q**, this sequence happens:

1. **Capture:** The Background script immediately takes a high-quality screenshot of your current tab.
2. **Extract:** The Content script simultaneously extracts any text it can find on the page.
3. **The Fallback Chain:** The Background script starts the "AI Relay Race":
  - **Attempt 1 (Gemini):** It tries Google Gemini first.
  - **Attempt 2 (Groq Vision):** If Gemini is busy or slow, it immediately switches to Groq using the Llama 4 Scout model.
  - **Attempt 3 (Groq Text):** If the screenshot is too complex, it sends just the text to a massive 70B model for a final attempt.
4. **Formatting:** It tells the AI exactly how to reply in JSON format.

## 📄 AI Structured Output (5-Word Headings)

Code Location: `background/background.js` (inside the `prompt` variable)

The AI generates structured responses because we send a **System Prompt** in every request.

### The Instruction Code:

```
const prompt = `You are an expert educational AI. Solve the following MCQ. Provide ONLY a valid JSON response in this exact format:
{
  "correctOption": "A",
```

```
"confidence": 90,  
"title": "5-word topic title of this question",  
"justification": "Concise explanation in 50-80 words..."  
};
```

The AI summarizes the question into a **5-word title** (e.g., "Principles of Quantum Mechanics") based on this instruction.

---

## Stealth & Anti-Cheat Bypass

Code Location: *content/content.js* (inside *enableAntiCheatBypass*)

1. **Shadow DOM:** The popup exists in a "secret layer" that the website's code cannot detect.
2. **Event Neutralization:** The extension intercepts "Tab Switch" signals. When you look at the AI popup, the website never knows you looked away from the exam.
3. **CSS Injection:** It forces the website to allow right-clicking and text selection.